

Enterprise ETL Pipeline & Data Warehouse Synchronizer

Project Report

Prepared for: Python Development Project 1
Report date: 8 September 2026

Executive Summary

This project implements a reusable enterprise-style extract, transform, and load (ETL) foundation. It retrieves operational data from Stripe and Salesforce, stores the unmodified payload for auditability, validates records, standardizes them with Pandas, and prepares the resulting data for incremental loading into PostgreSQL or Snowflake. Apache Airflow orchestrates the scheduled workflow and makes its execution observable and repeatable.

The delivered starter repository includes source code, a daily Airflow DAG, Docker Compose development infrastructure, environment-variable templates, and automated tests. It intentionally uses placeholder credentials only; no real Stripe keys, Salesforce tokens, or customer data are included.

Project Objectives

- Consolidate data from multiple third-party business APIs into one analytics-ready warehouse flow.
- Support API pagination and predictable retry-ready HTTP request patterns.
- Preserve raw JSON data before transformation so records can be investigated or replayed.
- Validate expected data types and reject malformed records before they reach analytical tables.
- Transform source-specific fields into consistent, warehouse-ready records.
- Enable daily scheduling, monitoring, and future incremental synchronization through Airflow.
- Provide a local, reproducible development environment using Docker and PostgreSQL.

Scope and Deliverables

Deliverable	Implementation status	Purpose
Stripe extractor	Implemented	Requests payment-intent data and follows paginated responses.
Salesforce extractor	Implemented	Runs a SOQL query and follows nextRecordsUrl pages.
Raw-data storage	Implemented locally	Writes source payloads as JSON; designed for later S3 integration.
Validation and transformation	Implemented	Pydantic validates core payment fields; Pandas creates normalized columns.
Airflow DAG	Implemented	Defines a daily extract → transform → load workflow.
PostgreSQL/Docker setup	Implemented	Provides a local warehouse database and Airflow runtime.
Automated test	Implemented	Checks amount conversion and timestamp transformation.
Warehouse MERGE/upsert	Extension required	The DAG contains a safe placeholder for staging and target-table loading.

Deliverable	Implementation status	Purpose
Production secrets/alerts/CI-CD	Extension required	Recommended before a real production release.

Architecture

Data flow: Stripe API and Salesforce REST API → raw JSON landing layer → schema validation and Pandas transformation → warehouse staging tables → dimensional/fact tables for analytics.

The landing layer separates data collection from business modeling. This design preserves the original payload, supports debugging and replay, and reduces the risk that a transformation defect irreversibly loses source information.

The workflow is designed to operate as a daily job. The Airflow DAG represents three logical tasks: extract data from APIs, transform it into a standardized dataset, and load it to a target warehouse. A future implementation should persist a watermark such as the latest successful source timestamp and retrieve only records created or updated after that point.

Technology Stack

Layer	Technology	Role
Programming	Python 3	Pipeline implementation and automation
Extraction	requests; Stripe API; Salesforce REST API	Authenticated API requests and pagination
Data processing	Pandas	Tabular transformations and timestamp/amount normalization
Validation	Pydantic	Schema validation and malformed-record handling
Orchestration	Apache Airflow	Daily DAG scheduling, retries, logs, and task monitoring
Warehouse	PostgreSQL; Snowflake-ready design	Staging, upserts, and analytical storage
Containerization	Docker Compose	Repeatable local development environment
Testing	pytest	Automated transformation validation
Optional raw storage	Amazon S3 / AWS S3	Durable data-lake landing zone

Core Implementation

The Stripe extractor uses an authenticated Bearer token and requests payment-intent records in batches. It checks the API response for additional pages and continues until no more records remain. The Salesforce extractor similarly handles paginated SOQL responses through the `nextRecordsUrl` field.

A Pydantic Payment model requires an identifier, non-negative amount, currency, creation timestamp, and status. Invalid items are logged and skipped rather than silently entering the analytical pipeline. The transform stage converts minor currency amounts to major-unit amounts, converts UNIX timestamps to timezone-aware timestamps, and retains a concise normalized set of fields.

The current Airflow DAG has the identifier **enterprise_etl_daily**, a daily schedule, no historical catch-up, and tags for ETL and warehouse operations. It chains the extract, transform, and load tasks in the correct dependency order.

Illustrative normalized output

Source field	Normalized field	Example
id	id	pi_1
amount	amount / amount_major	1250 / 12.50
created	created_at	UTC timestamp
status	status	succeeded

Security and Credentials

No real secrets are embedded in the project. The `.env.example` file contains placeholders such as `STRIPE_API_KEY=replace_me` and `SALESFORCE_ACCESS_TOKEN=replace_me`. A developer copies this file to a private `.env` file and supplies their own credentials locally.

For Stripe, the project expects a secret key, initially a test-mode key. For Salesforce, a production-grade deployment should use OAuth client credentials from a dedicated connected/external app and request short-lived access tokens programmatically. The access token should not be hard-coded into the DAG.

The `.env` file must be excluded from version control. For shared or production environments, credentials should reside in Airflow Connections, AWS Secrets Manager, or an equivalent managed secret store. The integration user should have least-privilege permissions and API keys should be rotated when exposure is suspected.

Testing and Quality Controls

The included pytest test verifies that the transformation turns an integer amount in minor units into the correct major-unit value and retains the expected currency. The project should be expanded with tests for API pagination, invalid schemas, duplicate source identifiers, null handling, watermark behavior, and warehouse upsert idempotency.

Recommended data-quality checks include record-count comparisons, unique primary-key checks, required-field null thresholds, valid currency/status checks, schema-drift detection, and reconciliation of extracted versus loaded records.

Deployment Procedure

- 1. Copy `.env.example` to `.env` and provide non-production test credentials.
- 2. Install Python dependencies from `requirements.txt`.
- 3. Start PostgreSQL and Apache Airflow using `docker compose up -d`.
- 4. Run `pytest` to confirm the included transformation test passes.
- 5. Open the Airflow web interface and enable `enterprise_etl_daily`.
- 6. Review task logs, raw JSON output, and the target warehouse before scheduling a production run.

Limitations and Next Steps

The repository is a functional foundation rather than a finished production data platform. The load task currently returns a result marker instead of writing to a physical staging and target schema. The next development stage should create warehouse tables, load transformed data in batches, and use an idempotent MERGE or PostgreSQL ON CONFLICT DO UPDATE operation keyed by source ID.

Priority next steps are: implement watermark persistence; add source updated-at filtering; upload raw payloads to S3 with date-partitioned paths; configure retries and exponential backoff; add Slack/email alerting; use Airflow Connections or a secrets manager; add CI checks; and document data ownership, retention, and recovery procedures.

Conclusion

This project demonstrates the core engineering patterns required for an enterprise API-to-warehouse synchronization service: secure configuration, paginated extraction, raw-data retention, validation, standardized transformation, orchestration, containerized development, and test coverage. With a warehouse upsert layer and production operational controls added, it can evolve into a dependable daily analytics pipeline.